

AALeC Quiz

Multiplayer-Quizsystem mit Embedded Hardware-Controllern



Pascal May
68763

Florian Merlau
81775

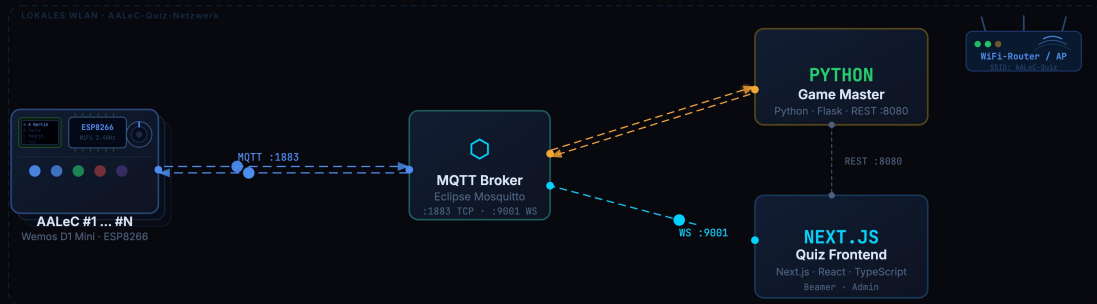
Projektidee & Anwendungsfall

 **Problem: Wissensabfragen in Vorlesungen sind umständlich**
Papierzettel, Handzeichen oder Online-Tools — schnelle, spontane Tests mit allen gleichzeitig sind in der Praxis aufwändig.

 **Anwendungsfall: interaktives Lernen**
Wissensabfragen und kleine Tests direkt in der Vorlesung, alle Teilnehmer antworten gleichzeitig mit ihrem Board.

 **Ziel: ein vollständiger eigener Stack**
Firmware, Kommunikation und Web-UI selbst gebaut, von Embedded bis Browser.

Vom Controller bis zum Beamer



AALeC-V3 — Wemos D1 Mini (ESP8266)

KOMPONENTE	FUNKTION IM QUIZ
OLED SH1106	Frage + A/B/C/D Antworten
Drehgeber + Taster	Auswahl bestätigen
WS2812B LEDs × 5	Richtig = grün, Falsch = rot
Potentiometer A0	Schätz- & Poti-Challenge
BME680 Sensor	Temperatur-Challenge
Piezo Beeper	Sound-Feedback
WiFi 802.11 b/g/n	MQTT über TCP/WS



GPIO-Belegungsplan

GPIO	PIN-NAME	KOMPONENTE & FUNKTION
GPIO 0	D3	Taster → Antwort bestätigen
GPIO 2	D4	WS2812B RGB-LEDs
GPIO 4	D2	I²C SDA → OLED
GPIO 5	D1	I²C SCL → OLED
GPIO 12	D6	Rotary Encoder Kanal A
GPIO 14	D5	Rotary Encoder Kanal B
GPIO 15	D8	Piezo-Beeper (PWM)
A0 (ADC)	A0	Potentiometer (10-Bit)

I²C

2-Draht-Bus (SDA + SCL)
OLED SH1106 · Adresse 0x3C
BME680 Sensor · Adresse 0x76

ADC

Einziger Analogeingang des ESP8266
10-Bit Auflösung (0–1023)
Liest Poti-Position für Schätzfragen

Digitale GPIOs

Encoder per Hardware-Interrupt
Beeper per PWM-Frequenz
LEDs per WS2812B-Protokoll (1-Wire)

Display-Zustände & UI-Konzept



Code-Aufbau & Software-Architektur

C++ 1. Firmware

src/firmware/Controller

main.cpp

Initialisierung und Haupt-Loop (Superloop). Verteilt Events je nach Spielzustand an die Untermodule.

display.cpp

Umfasst alle `showState()`-Renderfunktionen zur pixelgenauen OLED-Zeichnung im Hauptloop.

network.cpp

Verwaltet die WiFi-Verbindung und die blockierungsfreie MQTT-Reconnect-Schleife.

Py 2. Game Master / API

src/backend

game.py (GameMaster)

Zentrale Logik-Klasse. Verwaltet Spielerlisten, MQTT-Events und die State Machine des Quizzes.

scoring.py

Berechnet Punkte in Echtzeit: $\text{Punkte} = \text{Basis (1000)} + \text{Zeitbonus (max. 500)} \times \text{Streak-Bonus}$.

app.py (Flask API)

Stellt REST-Endpoints bereit, über die das Admin-Frontend Fragensets laden und aktivieren kann.

TS 3. Frontend

src/frontend

Beamer-View (page.tsx)

Die Projektionsfläche der Lobby, Frageseiten und des Leaderboards für die gesamte Klasse.

Admin-Panel (/admin)

Erlaubt dem Spielleiter, den Ablauf manuell zu triggern, Fragensets hochzuladen und Spieler zu verwalten.

mqtt.js Integration

Abonniert den Broker direkt im Browser über WebSockets (Port 9001), um sofort auf Zustandsänderungen zu reagieren.

Verwendete Bibliotheken

C++ Firmware

`PubSubClient` MQTT-Kommunikation

`ArduinoJson` JSON serialisieren / parsen

`ESP8266WiFi` WLAN-Verbindung

`SSD1306Wire` OLED-Display (I²C)

`Adafruit_BME680` Temperatursensor

`NeoPixelBus` WS2812B RGB-LEDs

Py Backend

`Flask` REST-API

`paho-mqtt` MQTT-Client (Game Master)

`python-dotenv` Konfiguration via .env

TS Frontend

`Next.js / React` UI-Framework

`mqtt.js` MQTT über WebSocket (Port 9001)

`Tailwind CSS` Styling

MQTT-Kommunikation

SERVER → ALLE

SERVER

quiz/state

Aktueller Spielzustand (retained)

SERVER

quiz/question

Frage mit Antwortoptionen

SERVER

quiz/reveal

Richtige Antwort + Statistik

SERVER

quiz/scores

Aktuelles Leaderboard

SERVER

quiz/players

Online-Spielerliste (retained)

SERVER

quiz/answer_count

Live-Zähler abgegebener Stimmen



CLIENTS → SERVER

CLIENT

quiz/answer/<id>

Antwort + Zeitstempel

CLIENT

quiz/connect/<id>

Gerät registrieren (Boot)

CLIENT

quiz/disconnect/<id>

Last Will (bei Verbindungsverlust)

CLIENT

quiz/control

start / end / restart (Admin)

Device-ID: aAlec-<hex>

aus ESP8266 Chip-ID generiert

MQTT Broker: Eclipse Mosquitto in Docker



Eclipse Mosquitto 2

Open-Source MQTT Broker · läuft als Docker-Container

- Image: `eclipse-mosquitto:2` · direkt von Docker Hub
- Config via Volume-Mount: `mosquitto.conf` read-only eingehängt
- Healthcheck per `mosquitto_sub` — Game Master startet erst wenn Broker bereit

ZWEI LISTENER

TCP :1883 ESP8266 Firmware via PubSubClient

WS :9001 Browser-Frontend via mqtt.js (WebSocket)

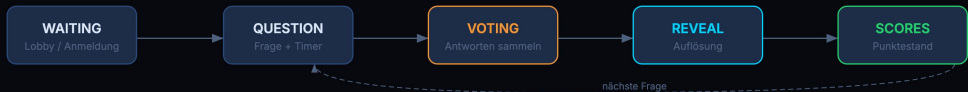
CONFIG `mosquitto.conf`

```
# TCP für ESP8266 Firmware listener 1883 protocol mqtt #  
WebSocket für Browser-Frontend listener 9001 protocol websockets  
allow_anonymous true
```

DOCKER `docker-compose.yml`

```
mosquitto:  
  image: eclipse-mosquitto:2  
  ports:  
    - "1883:1883" # TCP → ESP8266  
    - "9001:9001" # WS → Browser  
  volumes:  
    - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro  
  depends_on: #  
    game-master wartet auf healthy  
  healthcheck:  
    test: mosquitto_sub -t  
          $$SYS/# -C 1 -W 3  
    interval: 10s  
    timeout: 5s  
    retries: 3
```

Backend — Game Master & State Machine



FRAGETYPEN

Multiple Choice

Schätzfrage

↑ Höher / Niedriger

Poti-Challenge

Temperatur-Challenge

SCORING

1000 + **500** × **Streak**

Basis Zeitbonus Multiplikator

Fragetypen & Datenschema

mcq — Multiple Choice

```
{
  "text": "Idle-Pegel einer UART-Leitung?",
  "options": {
    "A": "LOW", "B": "HIGH",
    "C": "floating", "D": "Takt"
  },
  "correct": "B", "time_limit_s": 20
}
```

estimate — Zahlenwert schätzen

```
{
  "type": "estimate",
  "text": "millis()-Überlauf nach
          wie vielen Tagen?",
  "min": 1, "max": 100,
  "correct": 50, "unit": "Tage"
}
```

higher_lower — Höher oder niedriger?

```
{
  "type": "higher_lower",
  "text": "RS485-Reichweite > 100 m?",
  "reference": 100,
  "actual": 1200,
  "correct": "HIGHER", "unit": "m"
}
```

poti_target / temp_target — Hardware

```
{
  "type": "poti_target",
  "text": "Poti auf Zielwert einstellen!",
  "target_range": [15, 90], // zufällig
  "tolerance": 5,
  "time_limit_s": 25
}
```

Beamer-View — Was der Projektor zeigt

1 — WARTEN

localhost:3000/

AALeC Quiz

Warte auf Geräte...

Noch keine Geräte verbunden.
Noch 1 Spieler fehlt zum Starten.

Quiz starten

2 — FRAGE LÄUFT

localhost:3000/

Frage 3 / 1612s

Welche Programmiersprache wird für die ESP8266-Firmware eingesetzt?

A
C++

B
Python

C
Java

D
Rust

3 / 8 Antworten eingegangen

3 — AUFLÖSUNG

localhost:3000/

RICHTIG! ✓

Richtige Antwort: A — C++

A		55%
B		20%
C		15%
D		10%

Leaderboard wird aktualisiert...

1. BAABA — 1.200 Pkt	2. ANNA — 900 Pkt
----------------------	-------------------

Admin-Panel — Spielleiter-Steuerung

localhost:8080/admin

Quiz Admin

http://localhost:8080

MARKT

Starten

Abmelden

Neustart

TEILNEHMER (3)

BAABA
onlineANNA
onlineLEO
online

FRAGEN-SETS

▼ questions

questions.json

Exportieren

- Multiple Choice In welchem WLAN-Hotspot-Netzwerk müssen sich die AALeC-Clients anmelden?
- Multiple Choice Welcher Programmbereich wird genau einmal beim Starten des Controllers ausgeführt?
- Multiple Choice Was ist das Hauptterminal des 'Superloop'-Modells (Arduino standard loop)?
- Schätzfrage Wie viele Leitungen werden für die bidirektionale serielle UART-Kommunikation benötigt?
- Poll-Challenge Welche Sensor-Schnittstelle nutzt typischerweise eine Brückenschaltung?
- Multiple Choice Was passiert beim ESP8266, wenn eine blockierende Funktion länger als ein paar Sekunden läuft?
- Multiple Choice Was unterscheidet ein Thermoelement von einem NTC-Widerstand?

Probleme & Lösungen

01

Echtzeit-Sync aller Controller

PROBLEM

Mehrere ESP8266 müssen exakt denselben Spielzustand zeigen — ohne zentrale Taktung

LÖSUNG

State Machine publiziert Spielzustand — Controller reagieren rein event-getrieben

[quiz/state](#)

02

Namenseingabe ohne Tastatur

PROBLEM

Gerät hat nur Drehgeber + Taster + 128×64 OLED — kein Keyboard

LÖSUNG

Scroll-Liste per Encoder, Bestätigen per Klick — Backend verteilt Namen per MQTT

03

WLAN-Abbruch mitten im Spiel

PROBLEM

Kurzer WLAN-Ausfall — Controller verliert Zustand und hängt im falschen Screen

LÖSUNG

Reconnect-Loop mit Auto-Resubscribe — letzter State wird nach Reconnect neu gepusht

[quiz/connect](#)

04

128×64 OLED — zu wenig Platz

PROBLEM

Alle Spielzustände (Lobby, Multiple Choice, Balkendiagramm, Schätzung) leserlich auf nur 128×64 Pixel unterbringen — ohne den I2C-Bus den MQTT-Loop zu blockieren

LÖSUNG

Pixel-genaue Render-Funktion pro Spielzustand in `display.cpp` — I2C-Schreibzugriffe nur im `loop()`, nie in MQTT-Callbacks

GitHub Actions — Build, Push, Release



LIVE

Demo

Gemeinsam installieren & Multiplayer testen



Firmware flashen



Multiplayer starten



Quiz spielen

Projekt einrichten & starten

1 — Konfiguration

Firmware — `config.h`

```
// WLAN
#define WIFI_SSID "AALeC-Quiz"
#define WIFI_PASSWORD "12345678"

// MQTT-Broker
#define MQTT_BROKER "MacBook.local"
```

Backend — `.env`

```
MQTT_HOST=localhost
MQTT_PORT=1883
```

2 — Server starten

MQTT-Broker, Backend und Frontend starten:

```
docker-compose up -d
```

Startet drei Container:

<code>mosquitto</code>	MQTT-Broker :1883 / :9001
<code>game-master</code>	Backend / API :8080
<code>frontend</code>	Web-UI :3000

3 — Netzwerk

Mac als WLAN-Hotspot mit fester SSID:

SSID `AALeC-Quiz`

Passwort `12345678`

Broker `MacBook-Pro-von-Florian.local`

Controller verbinden sich automatisch sobald WLAN und Broker erreichbar sind.

Firmware flashen

flash.py — lädt Firmware automatisch von GitHub und erkennt den USB-Port

```
# Repo klonen
git clone https://github.com/XploradoX/ITS.git
cd ITS

# Abhängigkeit installieren
pip install esptool

# Flashen
python3 flash.py
```

Browser (kein Python nötig)

firmware.bin von GitHub Releases laden, dann im Browser flashen:

esptool.spacehuhn.com

Manuell (Terminal)

firmware.bin von GitHub Releases laden, dann:

```
# macOS / Linux
python3 -m esptool --port [port] --baud 115200 write_flash 0x0 firmware.bin

# Windows
python -m esptool --port COM3 --baud 115200 write_flash 0x0 firmware.bin
```



localhost: 3000

AALeC Quiz

Warte auf Geräte...

Noch keine Geräte verbunden.

Noch **1** Spieler fehlt zum Starten.

Quiz starten

Quiz Admin

http://localhost:8080

WARTET

0 online

▶ Starten

■ Beenden

↶ Neustart

TEILNEHMER (1)

Bro
aalec-982816

FRAGEN-SETS

questions 18

← Set auswählen oder neu erstellen

neues-set

+ Set

Was als Nächstes kommen könnte



01

Persistenz & Highscores

Datenbank für Spielhistorie, Highscores und Auswertung pro Spieler über mehrere Sessions



02

Fragen-Editor im Admin-Panel

Fragen direkt im Browser erstellen und bearbeiten statt JSON-Datei manuell zu pflegen



03

Anti-Cheat

Mehrfachabstimmung und Zeitmanipulation serverseitig erkennen und verhindern



04

Bilderfragen

Bilder direkt zur Frage auf dem Beamer anzeigen für visuellere Fragetypen



Fragen

**Danke für eure
Aufmerksamkeit**

Verwendete Tools & Ressourcen

HARDWARE & FIRMWARE

AALeC-V3 Hardware-Plattform	github.com/informatik-aalen/AALeC-V3
ESP8266 / Wemos D1 Mini	espressif.com/en/products/socs/esp8266
PlatformIO IDE	platformio.org
esptool.py (Flashen)	github.com/espressif/esptool
ArduinoJson	arduinojson.org

BACKEND, BROKER & FRONTEND

Eclipse Mosquitto	mosquitto.org
Python / Flask	flask.palletsprojects.com
Next.js / React	nextjs.org
MQTT Protokoll	mqtt.org
Projekt-Repository	github.com/XplorodoX/ITS